



TP 2 Deep Learning

25/09/2026

Nassim EL HADDAD

Exercice 1

Question 1 : Pourquoi utiliser `StandardScaler()` sur l'ensemble du dataset avant le split est une mauvaise pratique ?

```
$ python dataset.py  
Taille totale du dataset : 70000  
Shape features: torch.Size([64, 16])  
Shape labels: torch.Size([64, 1])
```

`StandardScaler()` calcule la moyenne et l'écart-type des données pour les normaliser. Si on l'applique avant de séparer le dataset, il utilise aussi les données de validation et de test, ce qui crée une fuite de données (*data leakage*). Il faut donc calculer le scaler uniquement sur les données d'entraînement puis appliquer la même transformation aux données de validation et de test.

Question 2 : Si le dataset fait 500 Go et ne tient pas en RAM, quelle classe utiliser ?

On utiliserait **`IterableDataset`**, car il permet de charger les données au fur et à mesure, sans avoir besoin de mettre tout le dataset en mémoire. C'est donc plus adapté pour un dataset très volumineux.

Exercice 2

2. Réponses pour le rapport

```
$ python train.py  
Precision: 0.7724 | Recall: 0.6736 | F1: 0.7196 | AUC: 0.8029
```

1. $l1_lambda = 0.1$ et $l2_lambda = 0$

Avec $l1_lambda = 0.1$, la pénalité L1 devient assez importante par rapport à la loss de classification. Le modèle cherche donc autant à réduire ses poids qu'à bien prédire les données. On peut alors observer que la loss diminue moins efficacement et que certains poids deviennent très petits, voire nuls.

La régularisation est donc trop forte : elle contraint trop le modèle et l'empêche d'apprendre correctement les relations présentes dans les données. Cela peut conduire à du sous-apprentissage (underfitting), avec de moins bonnes performances sur les données d'entraînement et de test.

2. Comment appliquer automatiquement la régularisation L2 ?

Avec SGD, on peut utiliser l'argument `weight_decay` pour appliquer directement une régularisation L2. Par exemple :

```
optimizer = optim.SGD(  
    model.parameters(),  
    lr=0.01,  
    weight_decay=1e-3  
)
```

Cela évite d'avoir à calculer nous-mêmes la pénalité L2 dans la boucle d'entraînement. `weight_decay` prend donc en charge cette régularisation directement au niveau de l'optimiseur.

3. Différence entre régularisation L1 et L2

L1 pénalise la somme des valeurs absolues des poids :

$$\sum |w|$$

Elle a tendance à pousser certains poids exactement vers zéro. Elle favorise donc des modèles avec moins de connexions réellement utilisées. On parle de sparsité.

L2 pénalise la somme des carrés des poids :

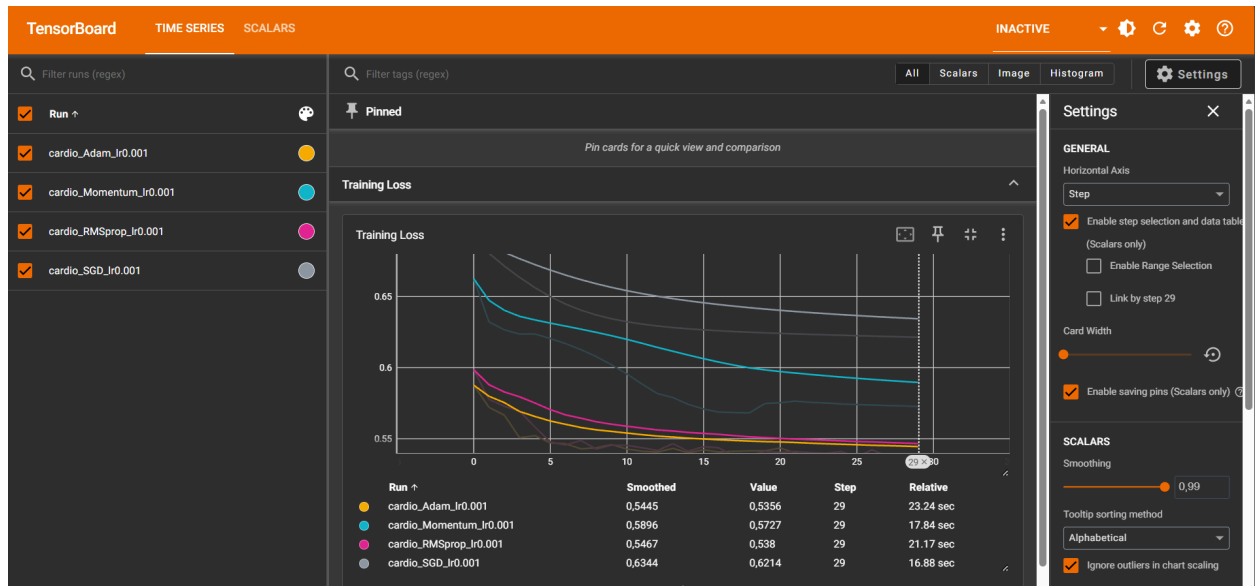
$$\sum w^2$$

Elle réduit généralement les poids vers de petites valeurs, mais sans nécessairement les rendre exactement nuls.

Régularisation	Pénalité	Effet principal
L1	$\sum$	w
L2	$\sum w^2$	Poids globalement plus petits

Donc L1 favorise la sparsité, tandis que L2 réduit progressivement l'amplitude des poids.

Exercice 3



2. Quel optimiseur converge le plus rapidement initialement ?

Avec cette configuration, Adam converge généralement assez rapidement au début de l'entraînement. RMSprop peut également converger rapidement. L'ordre peut cependant varier selon l'exécution, notamment à cause de l'initialisation aléatoire du réseau et de l'ordre des mini-batches. Il faut donc surtout se baser sur les courbes obtenues dans TensorBoard pour comparer les différents optimiseurs.

3. SGD vs Momentum

Avec le SGD, chaque mise à jour dépend uniquement du gradient calculé sur le batch actuel. Avec Momentum, l'optimiseur prend également en compte les mises à jour précédentes. Cela permet de garder une certaine inertie et d'accélérer la descente dans une direction cohérente.

En général, le Momentum permet :

- une convergence plus rapide ;
- moins d'oscillations ;
- une descente plus régulière.

On peut simplement retenir que le SGD réagit principalement au gradient actuel, tandis que SGD avec Momentum tient aussi compte de l'historique des gradients.

D'après les courbes obtenues dans TensorBoard, Adam converge le plus rapidement au début de l'entraînement, avec une forte baisse de la loss durant les premières époques. RMSprop suit une évolution assez similaire.

Pour SGD et Momentum, on observe que le Momentum permet une diminution plus rapide de la loss que le SGD classique. Cela correspond à son fonctionnement : en conservant une partie des mises à jour précédentes, il donne plus d'inertie à la descente et permet au modèle de progresser plus rapidement.

Exercice 4

```
$ python train.py  
Precision: 0.7506 | Recall: 0.7157 | F1: 0.7327 | AUC: 0.8018
```

1. Définition de Precision et Recall

La Precision indique, parmi les personnes que le modèle prédit comme malades, combien sont réellement malades.

$$\text{Precision} = \text{Vrais Positifs} / (\text{Vrais Positifs} + \text{Faux Positifs})$$

Le Recall indique, parmi toutes les personnes réellement malades, combien sont correctement détectées par le modèle.

$$\text{Recall} = \text{Vrais Positifs} / (\text{Vrais Positifs} + \text{Faux Négatifs})$$

Une Precision élevée signifie donc que le modèle fait peu de faux positifs. Un Recall élevé signifie qu'il fait peu de faux négatifs.

2. En médecine, faut-il privilégier Precision ou Recall ?

Pour détecter une maladie cardiovasculaire, il vaut mieux avoir un Recall élevé. Un faux négatif correspond à une personne qui est réellement malade, mais que le modèle considère comme non malade. Cela peut être problématique, car la maladie risque de ne pas être détectée.

À l'inverse, un faux positif signifie que le modèle considère une personne comme malade alors qu'elle ne l'est pas. Cela peut simplement conduire à faire des examens supplémentaires.



Dans le cas d'un dépistage, il est donc important de limiter les faux négatifs. Le choix du seuil dépend ensuite du contexte et de l'importance accordée aux faux positifs et aux faux négatifs.

3. À quoi sert l'AUC ?

La Precision, le Recall et le F1 sont calculés ici avec un seuil fixé à 0,5 :

```
all_preds_classes = (all_preds_probs > 0.5).astype(int)
```

L'AUC-ROC permet, elle, d'évaluer le modèle en faisant varier le seuil de classification. Elle donne donc une vision plus générale de sa capacité à distinguer les personnes malades des personnes non malades. Une AUC proche de 1 indique que le modèle distingue bien les deux classes, tandis qu'une AUC proche de 0,5 correspond à une performance proche du hasard.

L'AUC permet donc d'évaluer le modèle sans dépendre d'un seul seuil, comme celui de 0,5.